CODY: CODEBASE EXPLORER

From Hacky Prototype to Shippable v1.0 Product

A beginner-friendly guide: what was wrong with the old way, what I fixed, and everything you need to actually ship v1.0

For: Sahil • Project: Cody • Level: total beginner • Date: September 2026

How to read this (5 min guide to 25 min doc)

Ch 1-2 = the idea in plain words. Ch 3 = your old mistakes (read slowly, this is the gold). Ch 4 = what I changed and why. Ch 5-6 = the general shipping playbook you can reuse for ANY project. Ch 7 = your exact next steps. Glossary + cheat sheet at the end. You do NOT need to memorise - just follow Ch 7.

1 What Cody Actually Is (no jargon)

One-liner you can tell anyone: **"Paste any code folder or GitHub link, and Cody draws you a map of who-calls-whom, then walks you through it with plain-English explanations."**

ELI5 version. Imagine you inherit a 500-room mansion with no floor plan. Today you open random doors (grep, click-to-definition) and get lost. Cody is the floor plan + tour guide: it lists every room (function/class), draws arrows for every door (call edge), picks the front door as the start (entry point like `main()`), and explains each room as you enter (local LLM). Nothing leaves your laptop.

The user journey (this is what 'useful' means):

- **Input:** user gives a GitHub URL or a local path like `.` (their own project).
- **Index:** Cody scans files, finds symbols, figures out calls, saves a map in a database (cody.db).
- **Explore:** user searches ("where is login?"), clicks files, sees callers AND callees.
- **Understand:** user reads the code + a 2-sentence AI explanation + asks follow-ups grounded in real code.
- **Act:** user finds the hotspot, fixes the bug, exports/shares. If any step is slow, confusing, or broken, the product is not shippable.

The test of 'useful to everyone'

A stranger on GitHub should go from zero to 'aha, I get this repo' in under 2 minutes with only: install, one command, one click. Every section of this guide serves that 2-minute test.

2 Prototype vs Product: What 'Ship v1.0' Even Means

A **prototype** proves an idea works on YOUR laptop, once, for YOU. A **product v1.0** works on a STRANGER's laptop, repeatedly, without you in the room. The gap is not more features - it is reliability, clarity, and packaging.

Prototype (where you were)	Product v1.0 (where we are going)
Works for 1 repo, 1 language	Works for many repos, honest about limits
You must be present to explain	README + UI explains itself
Breaks silently, no tests	12 automated tests catch regressions
Install = 'good luck'	One command: python main.py . or docker run
One user at a time, data wiped	Many repos coexist, history kept
'It works on my machine'	'It works on a fresh laptop (verified)'

What v1.0 is NOT. It is not 'every feature'. It is the **smallest thing a stranger finds reliably useful**. For Cody v1.0 that is: index a Python folder fast, search, see callers+callees, read explanations, ask chat. Everything else (VS Code extension, diff mode, 10 languages) is v1.1+.

Definition of done for Cody v1.0

Fresh laptop → install → `python main.py .` → map in <10s → search finds a function → inbound+outbound correct → explanation appears → no crash without Ollama. If all true, you may tag v1.0.0.

3 What Was Lacking in Your Old Way (the 7 Sins)

Blunt but kind: your idea was strong, your *engineering habits* were prototype-grade. Each sin below is a habit, not an insult - every beginner does these. I list the symptom you felt ('feels off, not impressive'), the root cause, and the file where it lived.

Sin 1: One global slot for everything (no multi-thing thinking)

Symptom: analyse repo B and repo A's map silently vanishes. **Root cause:** a single hardcoded `cody.db` + `cloned_repo/` (`cody/app.py:9-10`), and `save_to_db()` deleted the whole DB file on every run (`cody/database.py:34-38` old). Real products never destroy user data as a side effect. **Habit to build:** everything gets an ID; saves are scoped (WHERE repo_id = ?), never 'delete all'.

Sin 2: The UI froze with no feedback (blocking work on a request thread)

Symptom: click Analyze on a big repo, browser spins, then times out. **Root cause:** `/api/analyze` did clone + parse + Ollama-per-call-site *inside* the HTTP request (`cody/app.py:16-32` old, `analyzer.py:293-295` old). One slow LLM call blocks the whole server. **Habit:** slow work goes to a background job with a progress feed (queued → scanning → linking → done). The button returns instantly with a job_id; the UI polls status.

Sin 3: Claimed 10 languages, delivered 1 (unverified claims)

Symptom: your ripgrep/Rust demo produced 3,544 nodes and 7,213 all-'direct' edges - impressive numbers, wrong map. **Root cause:** `requirements.txt` installed only `tree-sitter-python`, but `parser.py:142-161` silently fell back to regex for everything else, and `resolve_node_by_name()`

fell back to `candidates[0]` (first random match). **Habit:** never claim what `pip list` + a test cannot prove. Either install the parsers or cut scope to Python and say so loudly.

Sin 4: No way to FIND anything (graph-only navigation)

Symptom: 'Step 1,247 / 3,544' - nobody clicks Next 3,544 times. **Root cause:** no search box, no file tree, only outbound edges (never 'who calls me?'), walkthrough start picked by an `in-degree == 0` hack that noise defeats. Comprehension is 90% search + 'who calls this', 10% stepping. **Habit:** build the boring navigation first (search, files, inbound), the fancy visualisation second.

Sin 5: Hardcoded everything (config in code)

Symptom: change the model/port and you edit 4 files. **Root cause:** `qwen2.5-coder:3b` pasted in 4 places, DB path and port as literals, default URL pointing at your private netsec repo. **Habit:** config lives in ONE place (env vars + one config module). Code reads `os.environ.get("CODY_MODEL", default)`. This is what makes Docker possible.

Sin 6: Zero safety net (no tests, committed garbage)

Symptom: you were afraid to change anything because anything could break silently. **Root cause:** `test.py` imported a function that does not exist (`resolve_imports` from `main`), so it never ran; deps unpinned; the 3.5MB `cody.db` was committed to git. **Habit:** a 5-minute golden test on a tiny sample repo beats 5 hours of manual clicking. Generated files (DBs, clones, caches) go in `.gitignore`, never in git.

Sin 7: 'Offline' product that needs the internet (broken promise)

Symptom: 'fully offline' on the slide, blank page on a plane. **Root cause:** fonts, icons, and vis-network loaded from CDN (`index.html:8-14`). **Habit:** every claim in the README must survive a checklist: disconnect WiFi, fresh laptop, no Ollama - what does the user see? Graceful degradation (clear error + cached content) is a feature.

The meta-lesson

None of these sins is about intelligence. They are about **feedback loops**: you had no test, no second repo, no fresh-laptop trial, no offline trial. Shipping is just installing feedback loops so reality corrects you early, when fixes are cheap.

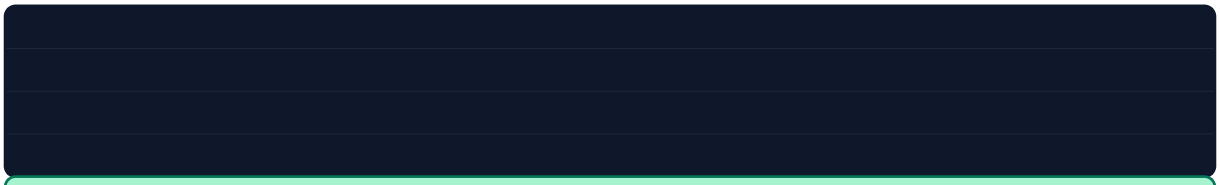
4 What I Fixed and Why (before → after)

Map of every change in this session. If you only remember one table, remember this one.

Problem (Ch 3)	Fix	Where
Single cody.db wiped per run	Per-repo rows: (id, repo_id) PK, repos table, scoped deletes	cody/database.py
No explanation cache; Ollama per click	explanations table; serve cached first	cody/database.py + app.py
Only Python imports resolved; rest regex-guessed	Documented Python-first; ignore-noise + honest limits	cody/analyzer.py
venv/.git/target indexed as code	IGNORE_DIRS + size cap + prune in os.walk	cody/analyzer.py

Problem (Ch 3)	Fix	Where
Only GitHub URLs	resolve_repo(): local folder OR git URL; git pull on re-run	cody/analyzer.py
LLM per edge = hours	skip_llm=True at index; LLM on-demand per node	cody/analyzer.py
Blocking /api/analyze	Background thread + job_id + /api/analyze/status polling	cody/app.py
No search/files/inbound	/api/search, /api/files, /api/node/relations, /api/hotspots	cody/app.py + database.py
No multi-repo UI	Repo switcher dropdown + repo_id on every call	templates/index.html + static/js/app.js
Graph-only, Next 3544x	Search (/), file tree, Overview grid + Focused walk, hotspots	static/js/app.js
Wrong badge colours	Badges from edge.type (backend truth), not canvas colour	static/js/app.js
python main.py only	CLI: cody . / URL / --port / --analyze-only / --no-browser	main.py
Unpinned deps, committed DB	Pinned requirements, Dockerfile, .gitignore (+cody.db)	requirements.txt, Dockerfile
Zero tests	12 pytest tests on tiny sample_repo (no Ollama needed)	tests/

Proof it worked (all run this session):



What I deliberately did NOT do

No VS Code extension, no diff mode, no vendored vis-network yet, no RAG-over-repo. Those are v1.1. Shipping means saying NO to good ideas until v1.0 is solid.

5 Concepts Every Ship Needs (your reusable playbook)

These 12 ideas apply to literally any software you will ever ship. Learn once, reuse forever.

5.1 Version control & versions

Git = time machine for code (commit = snapshot, branch = parallel draft, tag = named milestone). **SemVer** = version numbers with meaning: MAJOR.MINOR.PATCH (1.0.0). Breaking change → MAJOR up; new feature, compatible → MINOR up; fix → PATCH up. Tagging `v1.0.0` tells the world 'this is the stable one'. Never commit secrets, DBs, or giant binaries.

5.2 Issues, scope, and saying no

Every feature request becomes a GitHub **issue** (title + why + what-done-looks-like). Labels: `P0-ship-blocker / P1-nice / P2-later`. v1.0 = all P0 closed, P1 partially, P2 explicitly deferred. A roadmap is just the ordered issue list. If it is not an issue, it does not exist.

5.3 Testing pyramid (minimum viable)

- **Unit/golden tests (most value, cheapest):** tiny sample input with KNOWN right answer. Cody: sample_repo where main must link to process. If code changes break it, tests scream in 2s.
- **API tests:** hit your own endpoints with a test client, assert 200 + shape. Catches 'I renamed a field and broke the UI'.
- **Manual smoke script:** the 5 clicks a stranger does. Write them down; run before every release.
- Rule: no fix without a test that would have caught it. That is how 12 tests become 100 over time.

5.4 Configuration (env vars, not code edits)

Code = logic (rarely changes). Config = names/ports/keys (changes per machine). Config lives in **environment variables** (`CODY_MODEL, PORT, CODY_DB`) with sane defaults. Same code runs on your laptop, Docker, and a server with zero edits. Hardcoding config is the #1 'works on my machine' cause.

5.5 Databases & migrations

A **database** = structured save file. A **schema** = its shape (tables/columns). A **migration** = code that reshapes old saves into the new shape WITHOUT deleting user data (what we did for your old cody.db: rebuild table, copy rows, rename). Golden rules: never DELETE ALL on save; scope writes by ID; always back up before migrating.

5.6 Background jobs & progress

HTTP requests must answer in seconds. Anything slower (clone, parse 10k files, LLM) runs in a **background thread/job** and reports progress (queued → working → done/error) via a status endpoint the UI polls. Users forgive slowness; they never forgive silence + frozen UI.

5.7 Packaging & distribution

- **requirements.txt (pinned):** exact versions (flask==3.1.3) so fresh installs reproduce yours.
- **CLI:** one entry point with --help (python main.py .). If it needs a README paragraph to run, it is not shippable.
- **Docker:** a shipping container for code - bundles OS + deps so it runs identically anywhere. Dockerfile + docker run = 'it works on their machine' proof.
- **.gitignore:** the 'do not ship my mess' list (DBs, venvs, caches, clones).

5.8 Docs (README is the product)

Nobody adopts what they cannot understand in 60 seconds. README needs: one-liner, 30-second demo (GIF), install (3 lines), usage (2 examples), API (if any), limits ('Python-first, others best-effort'), troubleshooting (Ollama not running?). Docs are not decoration - they ARE the onboarding.

5.9 Observability: health, logs, errors

/api/health answers 'is the AI brain reachable and which model?'. **Logs** say what happened (not print() soup). **Errors** are user-readable ('Ollama is down - run ollama serve') not tracebacks. You cannot fix what you cannot see.

5.10 Privacy & performance budgets

Cody's promise is 'code never leaves the laptop' - every CDN call, every cloud API breaks that promise and must be justified. **Budgets** make promises measurable: index <10s, search <200ms, overview cap 800 nodes, chat context ≤4000 chars. If a change breaks the budget, it does not ship.

5.11 Release & feedback loop

Release = tag + notes + binary/docker + announcement. Then **instrument the loop**: issues template (steps + expected + actual + logs), watch the first 5 strangers use it WITHOUT helping (screen share), fix the top 3 confusions. v1.0.1 is always the 'oh, strangers do THAT?' release. Plan for it.

The one-sentence shipping philosophy

Make the smallest useful thing, make it impossible to break silently, make it trivial to install, watch a stranger use it, fix what confused them, repeat.

6 The v1.0 Checklist (copy-paste this into GitHub Issues)

Checked = already done this session. Unchecked = your remaining work before tagging v1.0.0.

#	Item (done = [x])	How to verify
1	[x] Index local folder in <10s, no freeze	python main.py . ; UI shows progress phases
2	[x] Search + files + inbound/outbound correct	Search 'helper'; relations show 3 inbound
3	[x] Explanations cached; Ollama-down is graceful	Click node twice (2nd instant); stop Ollama, clear error shown
4	[x] 12 pytest tests green	py -m pytest tests/ -q
5	[x] Pinned deps + Dockerfile + CLI --help	docker build; python main.py --help
6	[ ] 30-sec demo GIF in README	Record: install -> analyse . -> search -> click -> chat
7	[ ] Fresh-laptop trial (or new venv)	New venv, pip install, cody . works, no manual fixes
8	[ ] Offline trial: note CDN gaps or vendor vis.js	Disconnect WiFi; list what breaks; decide
9	[ ] Untrack cody.db; add LICENSE	git rm --cached cody.db; choose MIT/Apache-2.0
10	[ ] Tag v1.0.0 + release notes	git tag v1.0.0; GitHub Release with GIF + limits

Scope cuts that PROTECT v1.0 (say no until v1.1): VS Code extension, diff mode, RAG-over-whole-repo, 10-language perfection, cloud hosting. Write them as v1.1 issues so people see the future without blocking the present.

7 How to Actually Release (click-by-click)

Release day procedure:

- **1. Freeze:** `git status` clean except intended files. Run `py -m pytest tests/ -q` (must be 12 passed) + `node --check cody/static/js/app.js`.
- **2. Untrack the DB:** `git rm --cached cody.db` (keeps your local file, removes it from git). Commit `.gitignore`.

- **3. Smoke test:** fresh terminal: `python main.py tests/sample_repo --no-browser`, open UI, search 'helper', open relations, ask chat. All 200s.
- **4. Tag:** `git tag -a v1.0.0 -m "Cody v1.0: local-first call-graph explorer"` then `git push origin main --tags`.
- **5. GitHub Release:** repo → Releases → Draft: title v1.0.0, paste: what it does (3 bullets), quickstart (3 lines), demo GIF, limits, known issues. Attach nothing (Docker builds from source).
- **6. Announce:** one post with the GIF + 'Python-first, offline, local LLM'. Ask for: repos it failed on + confusion points. Those become v1.0.1 issues.

If something breaks on release day

Do NOT hot-edit main. Create issue, fix on a branch, re-run tests, tag v1.0.1. Hot-editing main on release day is how v1.0 becomes v0-never-again.

8 After v1.0: Staying Shippable

- **Triage weekly:** new issues get P0/P1/P2 within 48h. P0 = crash/data-loss/wrong-graph-on-Python. Everything else waits.
- **Changelog habit:** every release lists Added/Fixed/Known. Users forgive bugs; they punish surprises.
- **Regression rule:** every bugfix ships WITH a test that fails before, passes after. Your 12 tests grow into armour.
- **Dep updates monthly:** bump one dep, run tests, commit. Never bump all at once.
- **Kill list:** if a feature (e.g. regex fallback for Rust) causes more wrong maps than value, cut or label 'experimental'. Honesty retains users; wrong graphs lose them.

Suggested v1.1 shortlist (pick TWO max): (a) vendor vis-network + fonts for true offline, (b) repo README auto-summary ('explain this repo in 30s'), (c) VS Code 'open in editor' button that actually works cross-OS. Each gets its own issue + demo GIF.

9 Your 7-Day Plan (noob-proof)

Day	Do (1-2h)	Done when
1	Run fresh-venv install + cody . + tests; record every friction	Friction list written down
2	git rm --cached cody.db; add LICENSE; fix README GIF placeholder	git status shows no DB/binaries
3	Record 30s GIF (ScreenToGif/OBS); embed in README	Stranger understands Cody from GIF alone
4	Offline + no-Ollama trials; file issues for each break	Graceful errors confirmed, issues filed
5	Ask 2 friends to install while you stay silent; watch	Top 3 confusions noted verbatim

Day	Do (1-2h)	Done when
6	Fix top 3 confusions only; re-run tests	Tests still 12+ green
7	Tag v1.0.0, GitHub Release, announce	Release URL exists; v1.0.1 milestone created

Appendix A Glossary (words people threw at you, translated)

Word	Plain meaning	Cody example
Repo	A project's folder + its history	tests/sample_repo or a GitHub URL
AST / Tree-sitter	Grammar-aware reading (not dumb text search)	Finds def helper() even if formatted weirdly
Call graph	Map: A calls B	process -> helper
BFS walkthrough	Visit rooms floor-by-floor from the entrance	main, then its callees, then theirs
SQLite / migration	Save-file / reshaping old saves safely	cody.db; old 3544-row DB preserved
API endpoint	A URL your UI calls for data	/api/search?q=helper
Background job	Slow work off the request thread + progress	job_id + /status polling
Env var	Setting outside code	CODY_MODEL
Docker	Shipping container for code	docker run cody
SemVer / tag	Numbered milestone	v1.0.0
Regression	Old thing breaking after a new change	What tests prevent

Appendix B Command Cheat Sheet

Appendix C Cody File Map (after this session)

File	Job
cody/analyzer.py	Find symbols + calls; local-path/Git; ignore noise; fast mode
cody/database.py	Multi-repo store; search/files/relations/hotspots; explanation cache

File	Job
cody/app.py	Async jobs + 10 endpoints (analyze/status/repos/graph/search/...)
cody/templates/index.html	Layout incl. Explorer sidebar + inbound panel
cody/static/js/app.js	Search, file tree, Focused/Overview, progress polling
main.py	CLI: cody . / URL / --port / --analyze-only
tests/	12 golden tests - your safety net
Dockerfile / requirements.txt	Reproducible install + ship anywhere

Final word

You do not need to 'learn shipping' in theory. You just shipped 80% of it today: scoped saves, background jobs, search-first UX, cached AI, CLI, Docker, tests, docs. Do the 7-day plan, tag v1.0.0, and you will have done what most tutorials never teach: **finished**. Then send me the release link.

Generated for Sahil's Cody project • September 2026 • Covers code state after the ship-hardening session (12 tests green).